Team Software Engineering Project

Progress Report

<Intelligent Job Matching>

01286990 Team Software Engineering Project

Software Engineering Program

Department of Computer Engineering, KMITL

By

66011301 Yanin Sae Ma

66011244 Syril Tuladhar

Table of Contents

1. Project Description 2
   1.1. Problems 2
   1.2. Objective 3
   1.3. Project Plan for Intelligent Job Matching 3
      1.3.1. Requirement Analysis and Research 3
      1.3.2. System Design 3
      1.3.3. Development Phase 4
      1.3.4. Testing and Evaluation 4
      1.3.5. Deployment and Future Enhancement Planning 4
2. Software and Hardware Requirement 5
   2.1. User Requirement 5
      2.1.1. Functional Requirement 5
      2.1.2. Non-functional Requirement 5
   2.2. System Requirement 6
      2.2.1. Functional Requirement 6
      2.2.2. Non-functional Requirement 7
3. System Architecture 8
   3.1. System Architecture 8
      3.1.1. Frontend 8
      3.1.2. Backend 9
      3.1.3. Database 9
4. Use Case Analysis 10
5. Use Case Description 11
6. Sequence Diagrams (of 3 use cases) 19
7. Class diagrams 27

Team Software Engineering Project

<Intelligent Job Matching>

1. Project Description

1.1. Problems

In today's job market, both job seekers and employers face significant challenges in finding suitable matches efficiently. Traditional job portals rely heavily on keyword-based searches and manual filtering, which often leads to inaccurate or irrelevant job recommendations.

Job seekers frequently struggle to identify positions that align with their skills, experience, and career goals. Many applicants spend excessive time applying for jobs that are poorly matched to their profiles, resulting in frustration and reduced employment opportunities. Fresh graduates and individuals transitioning careers are particularly affected due to limited experience and unclear job requirements.

On the employer side, recruiters must process a large number of applications manually, which is time-consuming and prone to human error. Important candidate qualifications may be overlooked, while unsuitable applicants may advance in the hiring process. This inefficiency increases recruitment costs and delays hiring decisions.

Furthermore, existing systems often fail to consider deeper factors such as skill relevance, experience level, job preferences, and compatibility between candidates and job roles. The lack of intelligent matching mechanisms reduces the effectiveness of recruitment platforms and limits overall workforce productivity.

1.2. Objective

The primary objective of this project is to develop an **intelligent job matching system** that improves the accuracy and efficiency of matching job seekers with suitable job opportunities using artificial intelligence techniques.

The specific objectives of the project are as follows:

- To design a system that analyzes job seeker profiles, including skills, experience, education, and preferences.
- To enable employers to create and manage job postings by specifying job requirements, required skills, and preferred qualifications.
- To assist employers in identifying and ranking suitable candidates more effectively and accurately.
- To reduce the time and effort required for both job searching and recruitment processes.
- To provide a user-friendly platform that supports both job seekers and employers in the recruitment process.
- To enhance decision-making in recruitment by applying intelligent algorithms and structured reasoning.

1.3. Project Plan for Intelligent Job Matching

The development of the intelligent job matching system is carried out through several structured phases to ensure systematic progress and high-quality outcomes.

1.3.1. Requirement Analysis and Research

In this phase, the team studies existing job matching platforms and identifies their limitations. Functional and non-functional requirements are gathered from the perspectives of both job seekers and employers. Research is conducted on machine learning, natural language processing (NLP), and recommendation systems relevant to job matching.

1.3.2. System Design

The system architecture is designed to support scalability and modularity. Key components include a user interface for job seekers and employers, a backend server for data processing, a matching engine, and a database for storing user

profiles and job postings. Data models and matching criteria are defined during this stage.

1.3.3. Development Phase

During development, the frontend is implemented to allow users to create profiles, post jobs, and view recommendations. The backend is developed to handle data processing, matching logic, and API communication. Intelligent matching algorithms are implemented to compare job requirements with candidate profiles.

1.3.4. Testing and Evaluation

The system undergoes extensive testing to verify functionality, accuracy, and performance. Test cases are designed to evaluate matching results, system reliability, and user experience. Feedback is collected and used to improve system performance.

1.3.5. Deployment and Future Enhancement Planning

After successful testing, the system is prepared for deployment. The project also considers future enhancements such as real-time recommendations, skill gap analysis, and integration with external job platforms.

2. Software and Hardware Requirement

2.1. User Requirement

2.1.1. Functional Requirement

1. The system must allow users, job seekers and employers, to create, edit and update their personal profiles.
2. The system must allow job seekers to manually enter their personal information, including skills, education, experience, and job preferences.
3. The system must allow job seekers to upload a resume to automatically populate their profile.
4. The system must allow job seekers to view a list of recommended job positions ranked according to their suitability scores.
5. The system must allow job seekers to view detailed explanations of job matching results, including matched skills and missing skill gaps.
6. The system must allow employers to create, edit, and delete job postings by specifying job descriptions, required skills, and preferred qualifications.
7. The system must allow employers to view candidate profiles and evaluate their suitability based on system-generated matching scores.
8. The system must allow employers to view and manage the list of candidates matched to their job postings.
9. The system must allow both job seekers and employers to update their personal and account information when changes are required.
10. The system must allow users to communicate with each other through a messaging feature when both parties are interested in a job opportunity.

2.1.2. Non-functional Requirement

1. The user interface must be simple and intuitive, ensuring that users can easily access key features such as profile creation, job posting, and job recommendation results.
2. The system must be fully responsive, providing a user-friendly experience across various devices including smartphones, tablets, and laptops.
3. When accessing the system, the main dashboard should load and become visible within 2 seconds under normal operating conditions.

4. Navigation between pages must be smooth, and page transitions should take no longer than 2 seconds.
5. The system must present job matching results in a clear and readable format, including match scores, skill matches, and missing skill gaps.
6. The system must provide visual highlighting of key information in job descriptions to enhance user comprehension and readability.
7. The system must be able to support at least 500 concurrent users without significant performance degradation.
8. The system must ensure data security and privacy by protecting user information such as resumes, personal details, and job postings from unauthorized access.
9. The system should be available 24 hours a day, 7 days a week, except during scheduled maintenance periods.
10. The system must provide clear and informative error messages when users perform invalid actions, such as uploading unsupported file formats or submitting incomplete information.
11. The system must be able to run smoothly on a system with at least 4 GB of RAM and a standard web browser.

2.2. System Requirement

2.2.1. Functional Requirement

1. The system must allow job seekers to create new personal profiles, edit existing profile information, and update their details when necessary.
2. The system shall allow job seekers to manually enter personal information, including skills, education, experience, and job preferences.
3. The system must allow job seekers to upload a resume to automatically populate their profile.
4. The system must allow job seekers to view recommended job listings ranked according to suitability and matching scores.
5. The system must allow employers to create new job postings, edit existing job postings, and delete job postings when required.
6. The system must allow employers to specify job descriptions, required skills, and preferred qualifications for each job posting.
7. The system must allow employers to analyze and view matched candidates generated by the intelligent job matching process.

8. The system must allow employers to view detailed candidate profiles, including skill matches, missing skill gaps, and match scores.
9. The system must allow job seekers and employers to communicate with each other through a messaging feature when mutual interest in a job is established.
10. The system must allow both job seekers and employers to edit their personal and account information whenever changes are required.

2.2.2. Non-functional Requirement

1. The system must be developed as a web-based application to ensure accessibility across multiple devices and platforms, including desktops, tablets, and smartphones.
2. The system must use JavaScript with the React framework for frontend development to provide a responsive, modular, and interactive user interface.
3. The system must use Python 3 as the primary backend programming language due to its simplicity, extensive ecosystem, strong community support, and strong support for Object-Oriented Programming (OOP).
4. The system must implement the backend using the Flask framework, chosen for its lightweight nature, flexibility, and suitability for building RESTful web services.
5. The system must follow a REST API architecture to enable efficient and structured communication between the frontend and backend components.
6. The system must use MySQL as the database management system, managed through MySQL Workbench, to ensure reliable, scalable, and persistent data storage.
7. The system must incorporate Prolog as a logic programming language to support rule-based reasoning and intelligent decision-making in the job matching process.
8. The system must enable interaction between the Python backend and the Prolog reasoning engine to evaluate job suitability based on predefined rules and constraints.
9. The system must manage source code using Git as the version control system, with GitHub used as the hosting platform to support team collaboration, version tracking, and maintenance.

10. The system must implement a secure authentication mechanism using JWT (JSON Web Tokens) to ensure secure user authentication and session management.

3. System Architecture

3.1. System Architecture

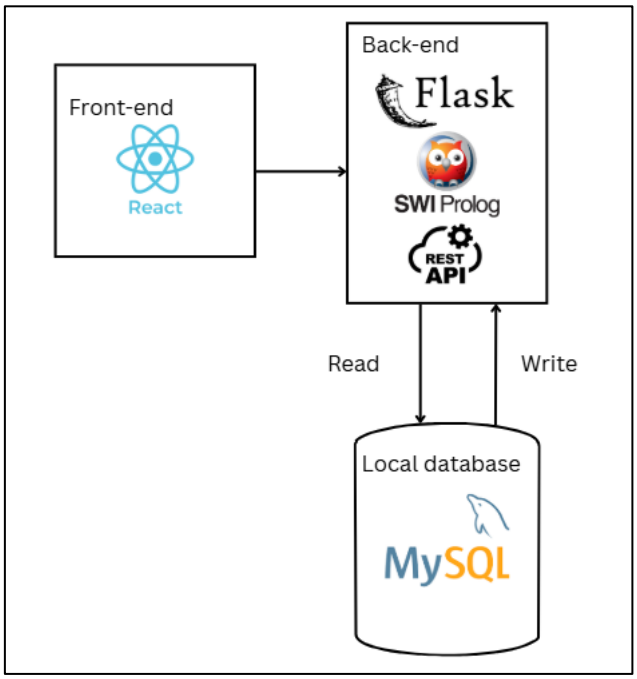


Diagram explain the interaction between different components of a system

3.1.1. Frontend

The front-end layer serves as the user-facing interface of the intelligent job matching system. It is developed using **React**, a popular JavaScript library for building interactive and component-based user interfaces. React enables the creation of dynamic and responsive web pages, allowing users to efficiently interact with the system through features such as profile creation, job posting, job searching, and viewing job matching results. Its component-based architecture improves code reusability and maintainability while providing a smooth user experience across different devices and screen sizes.

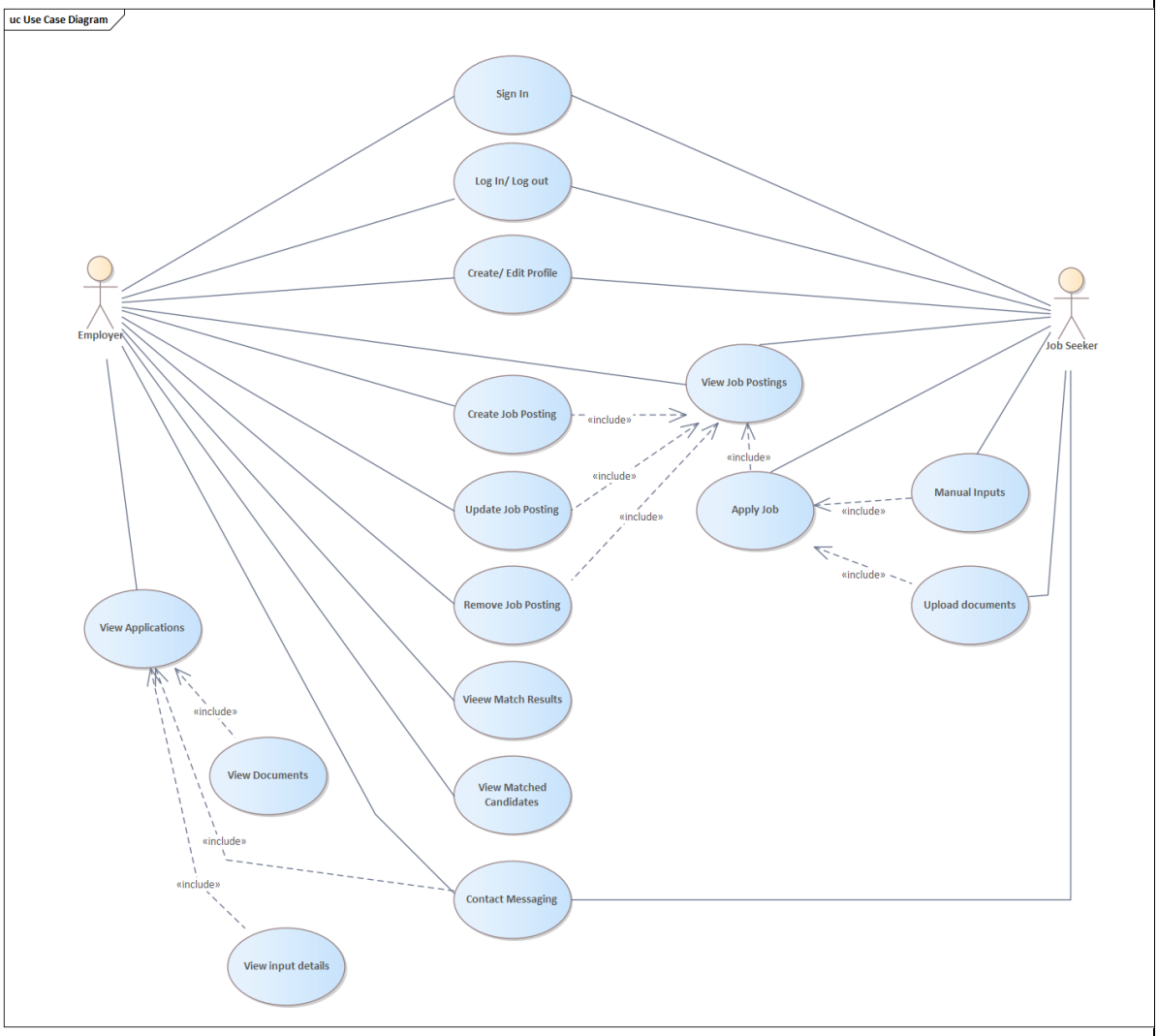
3.1.2. Backend

The backend of the system is implemented using **Flask**, a lightweight and flexible Python web framework suitable for building RESTful web applications. This layer is responsible for handling HTTP requests from the frontend, managing core application logic, coordinating communication between system components, and managing authentication. The backend exposes a set of **RESTful APIs** that enable structured communication between the frontend and backend using JSON format, supporting key operations such as user profile management, job posting, resume submission, and job matching requests. In addition, the backend retrieves and stores data in the database and integrates with the Prolog reasoning engine to perform intelligent job matching, returning processed results to the frontend in a clear and readable format.

3.1.3. Database

The system uses **MySQL** as its database management system to provide reliable and scalable persistent data storage. MySQL stores essential data such as user profiles, job postings, skill information, and job matching results. To interact with the database, the backend employs **SQLAlchemy**, a Python Object Relational Mapper (ORM) that enables database operations using Python objects instead of raw SQL queries. This abstraction improves data consistency, simplifies database management, and enhances long-term maintainability of the system.

4. Use Case Analysis



5. Use Case Description (for at least 3 use cases)**, each of which includes 1. Basic Course of Event, 2 Alternative Courses of Event, and 3. Exceptional Courses of Event)**

Use Case 1: Register / Login

Actor: Job Seeker / Employer

Description:
This use case describes how a user creates an account or logs into the intelligent job matching system to access system features.

1. Basic Course of Events

1. The user selects the “Register” or “Login” option on the homepage.
2. The system displays the authentication form.
3. The user enters required email and password.
4. The user submits the form.
5. The system validates the credentials.
6. The system creates a new account for registration or authenticates the user for login.
7. The system redirects the user to their dashboard.

2. Alternative Courses of Events

- **A1: User chooses Employer role during registration**
  The system assigns employer privileges and redirects to the employer dashboard.
- **A2: User chooses Job Seeker role during registration**
  The system assigns job seeker privileges and redirects to the job seeker dashboard.

3. Exceptional Courses of Events

- **E1: Invalid Credentials**
  If login credentials are incorrect, the system displays an error message and prompts the user to try again.
- **E2: Email Already Registered**
  If the email already exists during registration, the system notifies the user and prevents duplicate account creation.

Use Case 2: Intelligent Job Matching

Actor: Job Seeker

Description:
This use case describes how the system generates job recommendations using rule-based reasoning based on the job seeker's skills and profile information.

1. Basic Course of Events

1. The job seeker logs into the system.
2. The job seeker completes or updates their profile with skills and experience.
3. The job seeker selects "Find Matching Jobs."
4. The system retrieves the job seeker's skills from the database.
5. The system retrieves available job postings.
6. The system sends relevant data to the Prolog reasoning engine.
7. The Prolog engine evaluates job suitability using predefined rules.
8. The system calculates match scores and identifies skill gaps.
9. The system returns the ranked job list to the frontend.
10. The job seeker views the matching results.

2. Alternative Courses of Events

- **A1: Partial Match Found**
  The system displays jobs with lower match scores and highlights missing skills.
- **A2: Updated Profile Before Matching**
  If the user edits their skills before matching, the system recalculates results using updated information.

3. Exceptional Courses of Events

- **E1: No Job Matches Found**
  The system displays a message indicating no suitable jobs are available and suggests updating skills.
- **E2: Reasoning Engine Failure**
  If the Prolog engine fails to respond, the system logs the error and displays a temporary system error message.

Use Case 3: Create Job Posting

Actor: Employer

Description:
This use case describes how an employer creates and publishes a job posting within the system.

1. Basic Course of Events

1. The employer logs into the system.
2. The employer selects "Create Job Posting."
3. The system displays the job posting form.
4. The employer enters job title, description, required skills, and other relevant details.
5. The employer submits the job posting form.
6. The system validates the input data.
7. The system saves the job posting in the database.
8. The system confirms successful job creation.

2. Alternative Courses of Events

- **A1: Employer Edits Existing Job Posting**
  The employer selects an existing job and updates its details. The system saves the updated information.
- **A2: Employer Deletes Job Posting**
  The employer selects a job posting and deletes it. The system removes it from the database.

3. Exceptional Courses of Events

- **E1: Missing Required Fields**
  If required fields are empty, the system displays validation errors and prevents submission.
- **E2: Unauthorized Access**
  If a non-employer attempts to access job posting functionality, the system denies access and redirects to the appropriate page.

Use Case 4: Update Job Posting

Actor: Employer

Description:
This use case describes how an employer updates the details of an existing job posting.

1. Basic Course of Events

1. The employer logs into the system.
2. The employer navigates to "Manage Job Postings."
3. The system displays a list of the employer's posted jobs.
4. The employer selects a job to update.
5. The system displays the editable job details form.
6. The employer modifies job information (e.g., description, required skills).
7. The employer submits the updated form.
8. The system validates the input data.
9. The system updates the job record in the database.
10. The system confirms successful update.

2. Alternative Courses of Events

- **A1: Partial Update**
  The employer updates only selected fields (e.g., salary or required skills), and the system updates only modified attributes.
- **A2: Save as Draft**
  The employer saves the changes as a draft instead of publishing immediately.

3. Exceptional Courses of Events

- **E1: Invalid Input Data**
  If required fields are missing or incorrect, the system displays validation errors.
- **E2: Unauthorized Access**
  If a non-owner attempts to edit the job posting, the system denies access.

Use Case 5: Remove Job Posting

Actor: Employer

Description:
This use case describes how an employer deletes an existing job posting from the system.

1. Basic Course of Events

1. The employer logs into the system.
2. The employer navigates to "Manage Job Postings."
3. The system displays the employer's job postings.
4. The employer selects a job to remove.
5. The employer confirms the deletion request.
6. The system deletes the job posting from the database.
7. The system confirms successful removal.

2. Alternative Courses of Events

- **A1: Cancel Deletion**
  The employer cancels the deletion request before confirmation.
- **A2: Archive Instead of Delete**
  The employer chooses to archive the job posting instead of permanently deleting it.

3. Exceptional Courses of Events

- **E1: Job Not Found**
  If the job ID does not exist, the system displays an error.
- **E2: System Failure During Deletion**
  If a database error occurs, the system logs the error and informs the employer.

Use Case 6: Apply for Job

Actor: Job Seeker

Description:
This use case describes how a job seeker applies for a job posting through the system.

1. **Basic Course of Events**

1. The job seeker logs into the system.
2. The job seeker views job details.
3. The job seeker selects “Apply Job.”
4. The system checks whether the job seeker profile is complete.
5. The job seeker uploads a resume or confirms existing resume.
6. The system stores the application in the database.
7. The system confirms successful application submission.

2. **Alternative Courses of Events**

- **A1: Apply Using Saved Resume**
  The job seeker selects an already uploaded resume.
- **A2: Withdraw Application**
  The job seeker withdraws the application before employer review.

3. **Exceptional Courses of Events**

- **E1: Incomplete Profile**
  If required profile information is missing, the system prompts the user to complete it.
- **E2: Duplicate Application**
  If the job seeker already applied, the system prevents duplicate submission.

Use Case 7: Contact Messaging

Actor: Job Seeker / Employer

Description:
This use case describes how users communicate with each other through the messaging system.

1. **Basic Course of Events**

1. The user logs into the system.
2. The user selects a conversation or recipient.
3. The system displays the messaging interface.
4. The user types a message.
5. The user sends the message.
6. The system stores the message in the database.
7. The recipient receives the message notification.

2. **Alternative Courses of Events**

- **A1: Reply to Existing Conversation**
  The recipient responds to the message within the same conversation thread.
- **A2: Attach Resume or Job Link**
  The user attaches relevant information to the message.

3. **Exceptional Courses of Events**

- **E1: Empty Message**
  If the message content is empty, the system prevents sending.
- **E2: Unauthorized Messaging Attempt**
  If a user tries to contact someone without proper access, the system denies the action.

Use Case 8: View Application

Actor: Employer

Description:
This use case describes how an employer views applications submitted for a job posting.

1. **Basic Course of Events**

1. The employer logs into the system.
2. The employer selects a job posting.
3. The employer chooses “View Applications.”
4. The system retrieves submitted applications from the database.
5. The system displays applicant details and resume information.
6. The employer reviews application details.

2. **Alternative Courses of Events**

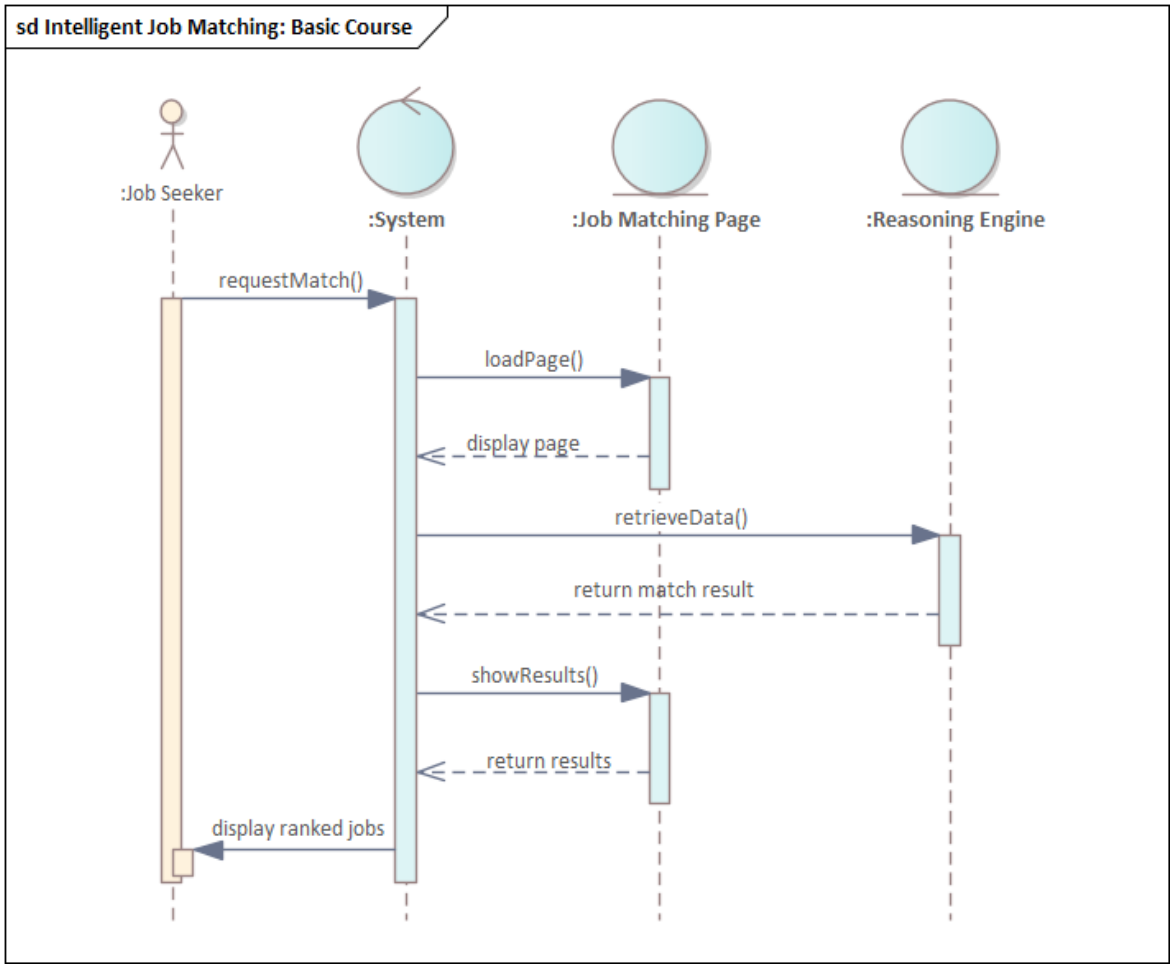
- **A1: Filter Applications**
  The employer filters applications based on match score or qualifications.
- **A2: Shortlist Candidate**
  The employer marks a candidate as shortlisted.

3. **Exceptional Courses of Events**

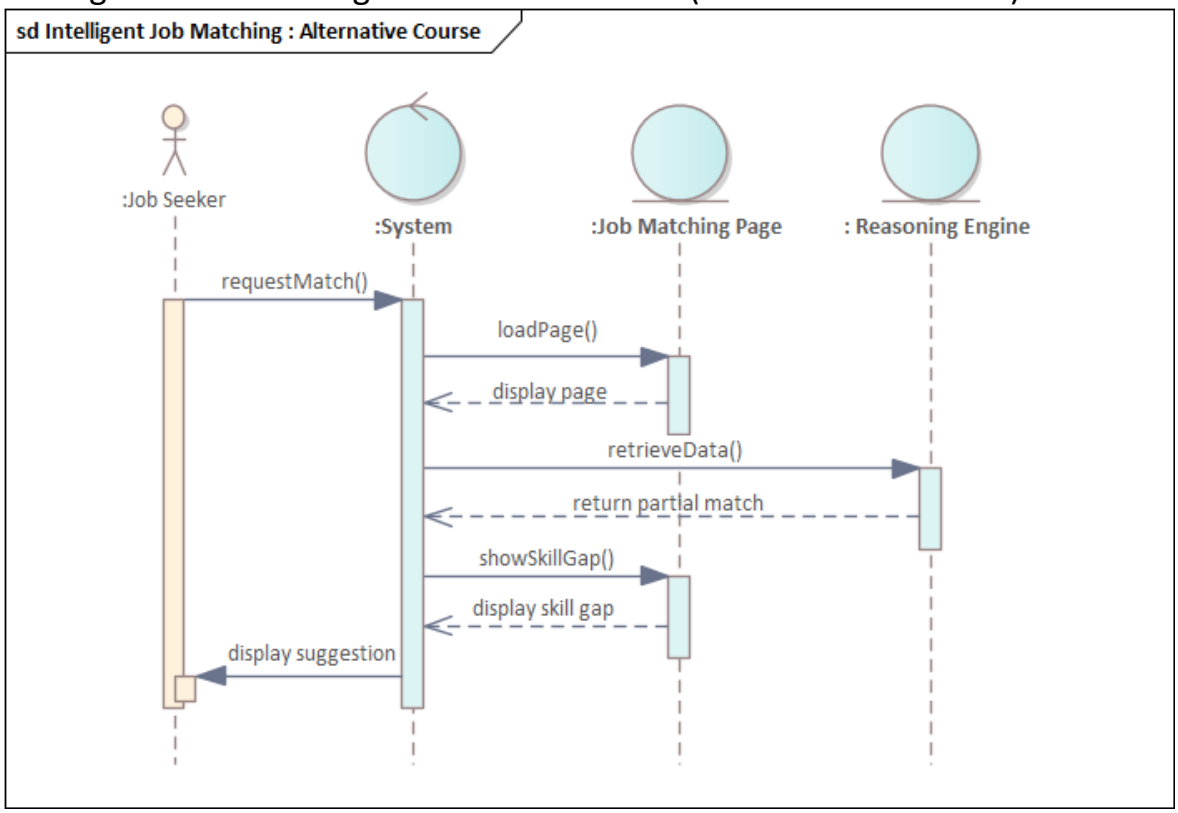
- **E1: No Applications Available**
  The system displays a message indicating no applicants.
- **E2: Data Retrieval Failure**
  If database access fails, the system shows an error message.

6. Sequence Diagrams (of 3 use cases)

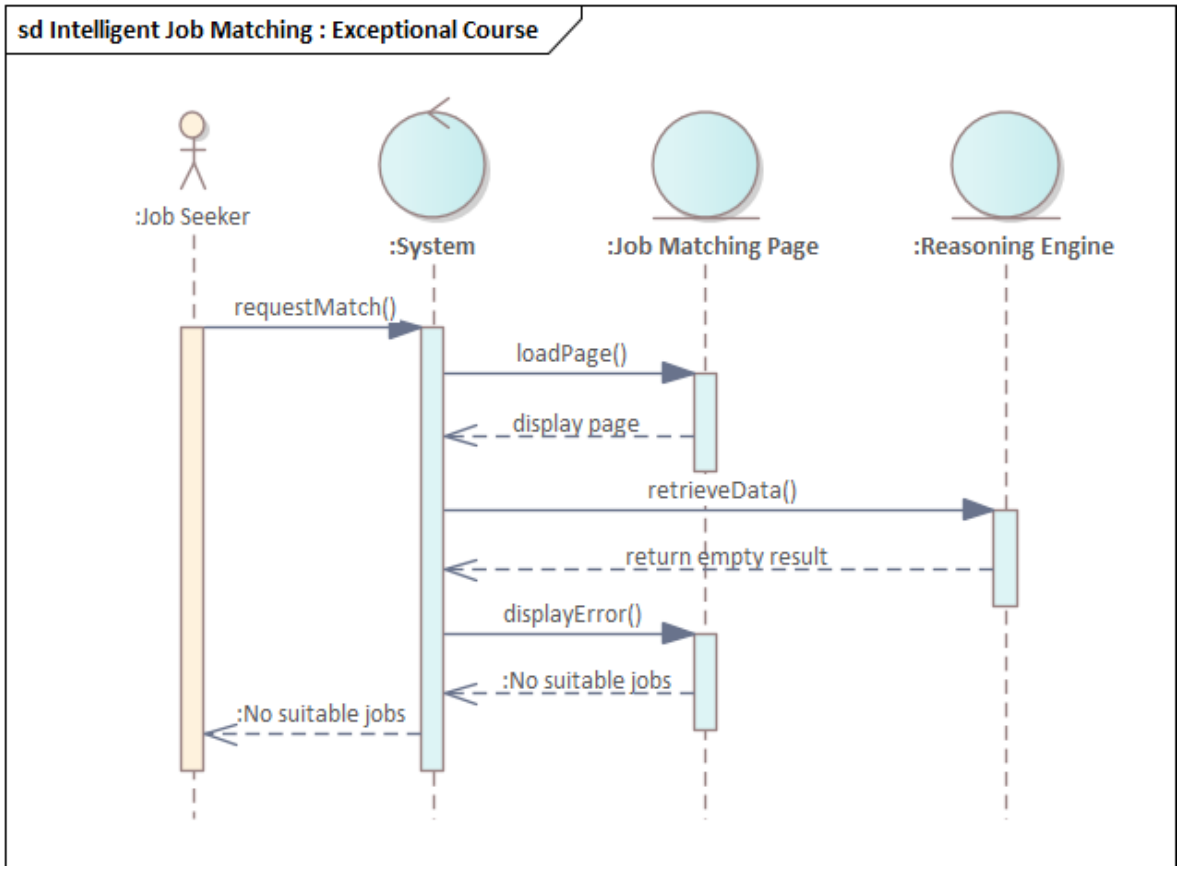
Intelligent Job Matching : Basic Course



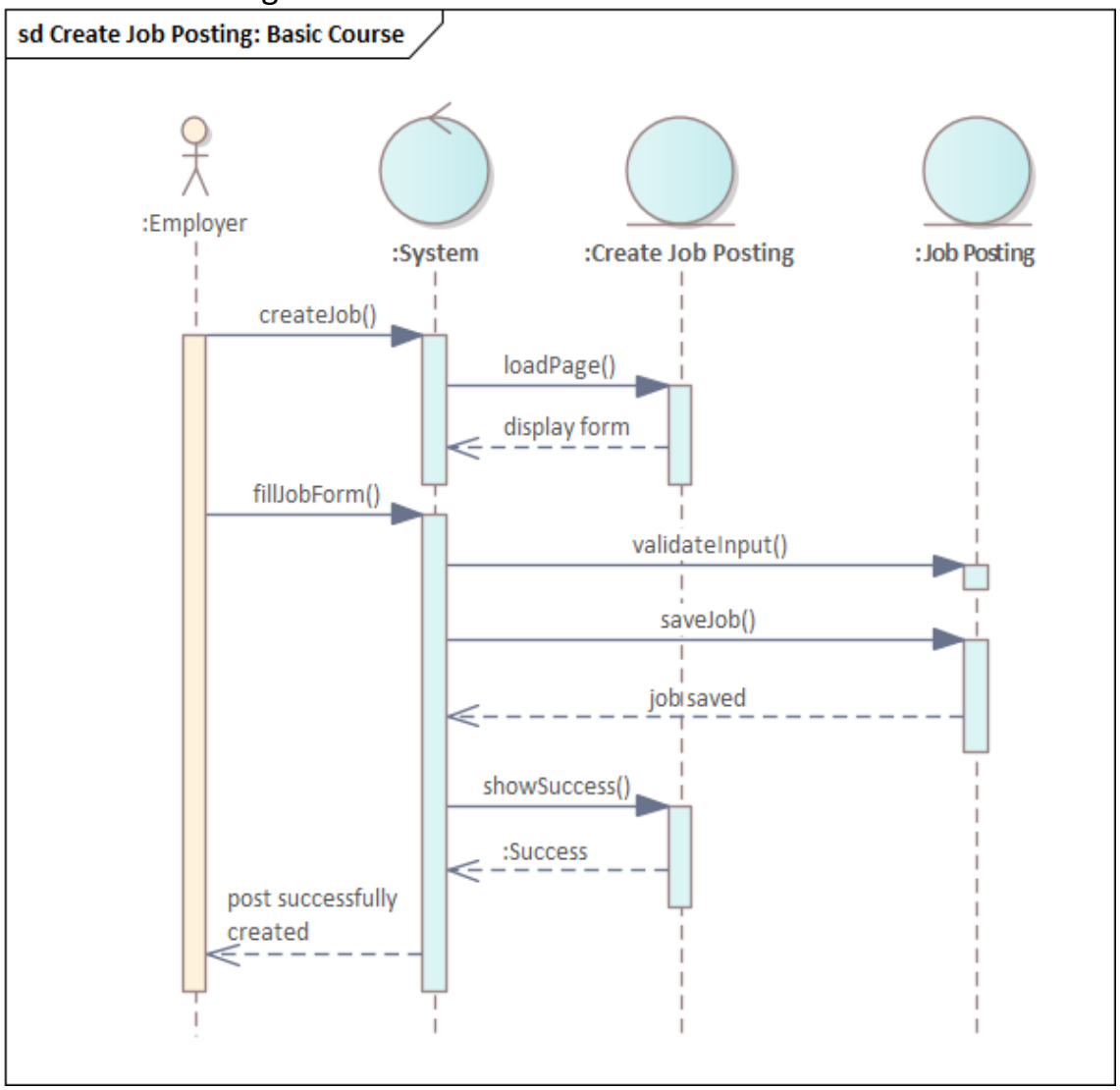
Intelligent Job Matching : Alternative Course (Partial Match Found)



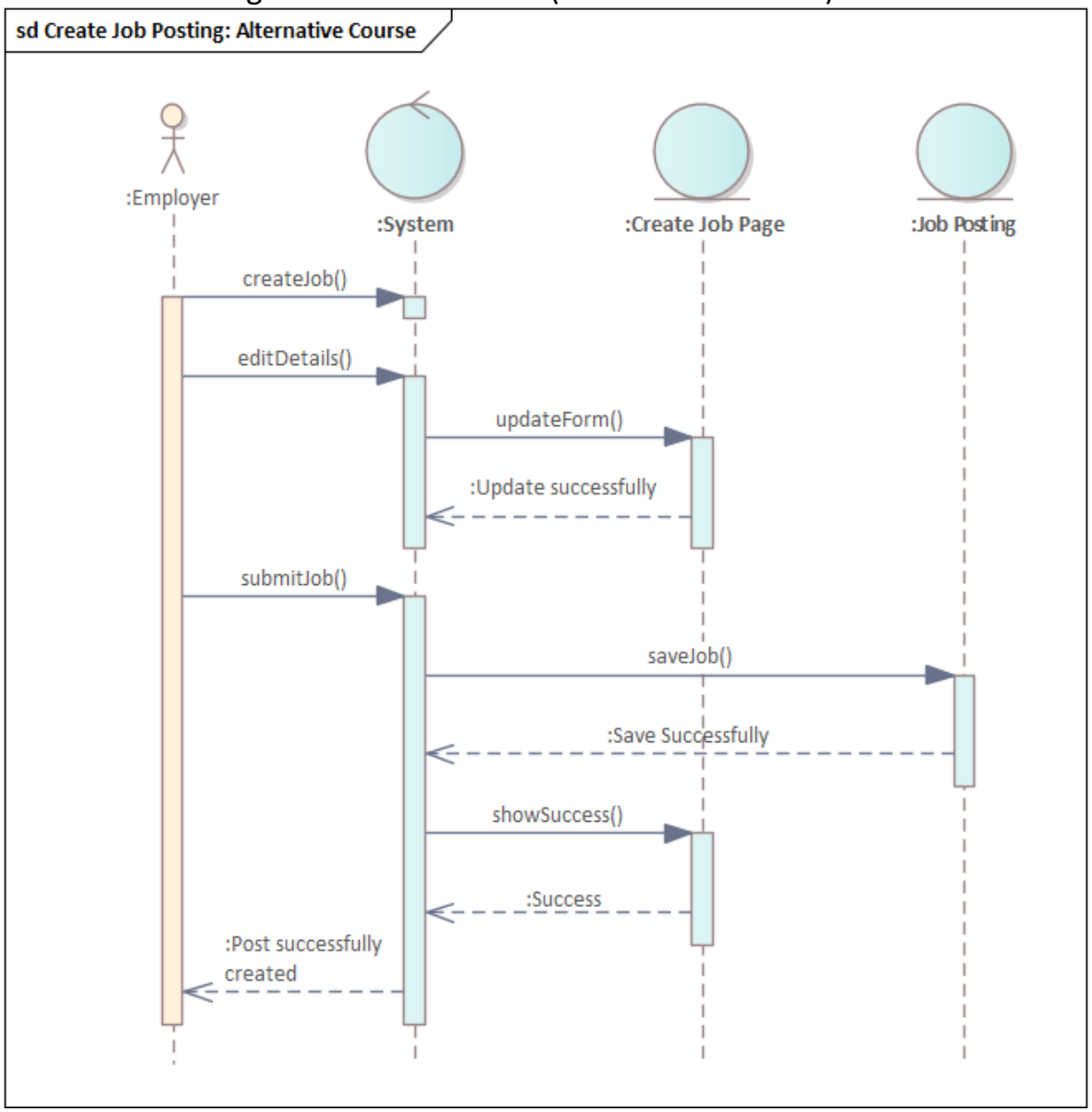
Intelligent Job Matching : Exceptional Course (No Match Found)



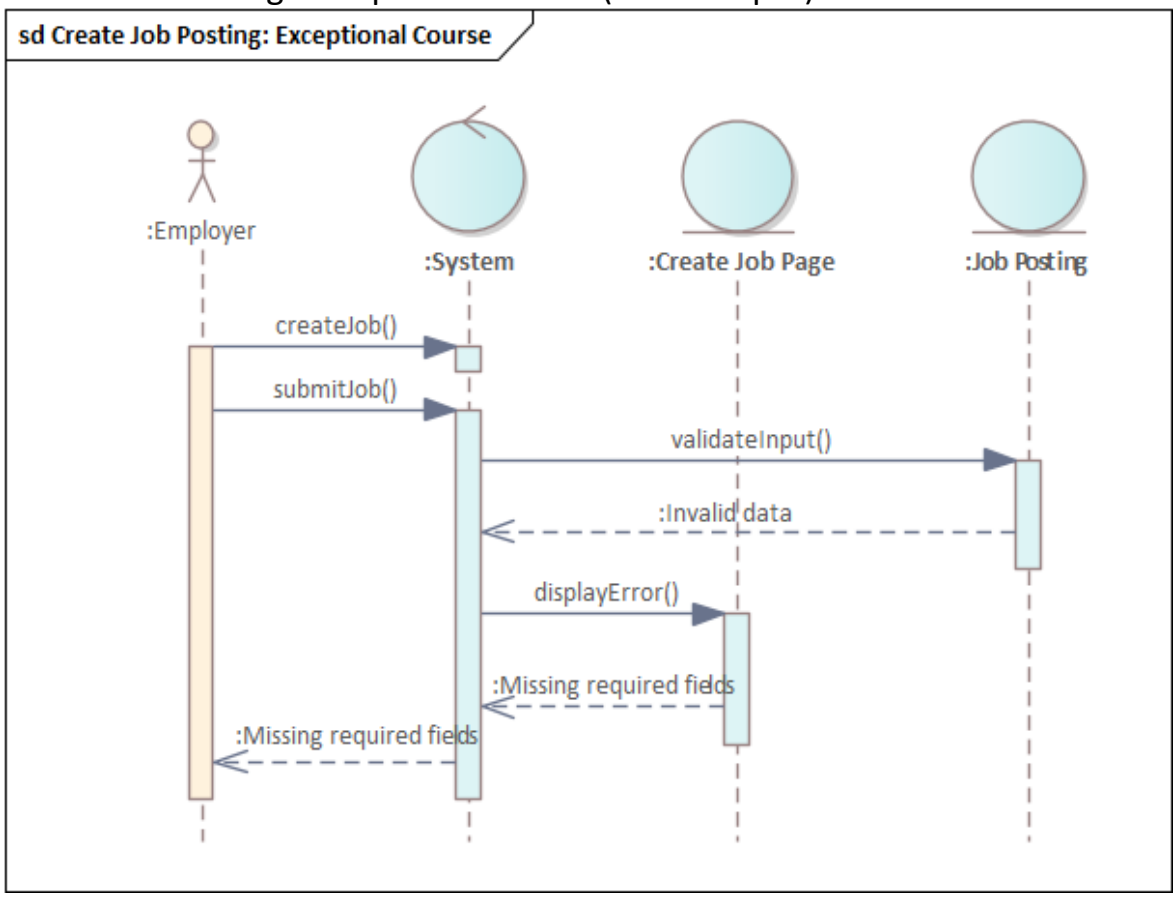
Create Job Posting: Basic Course



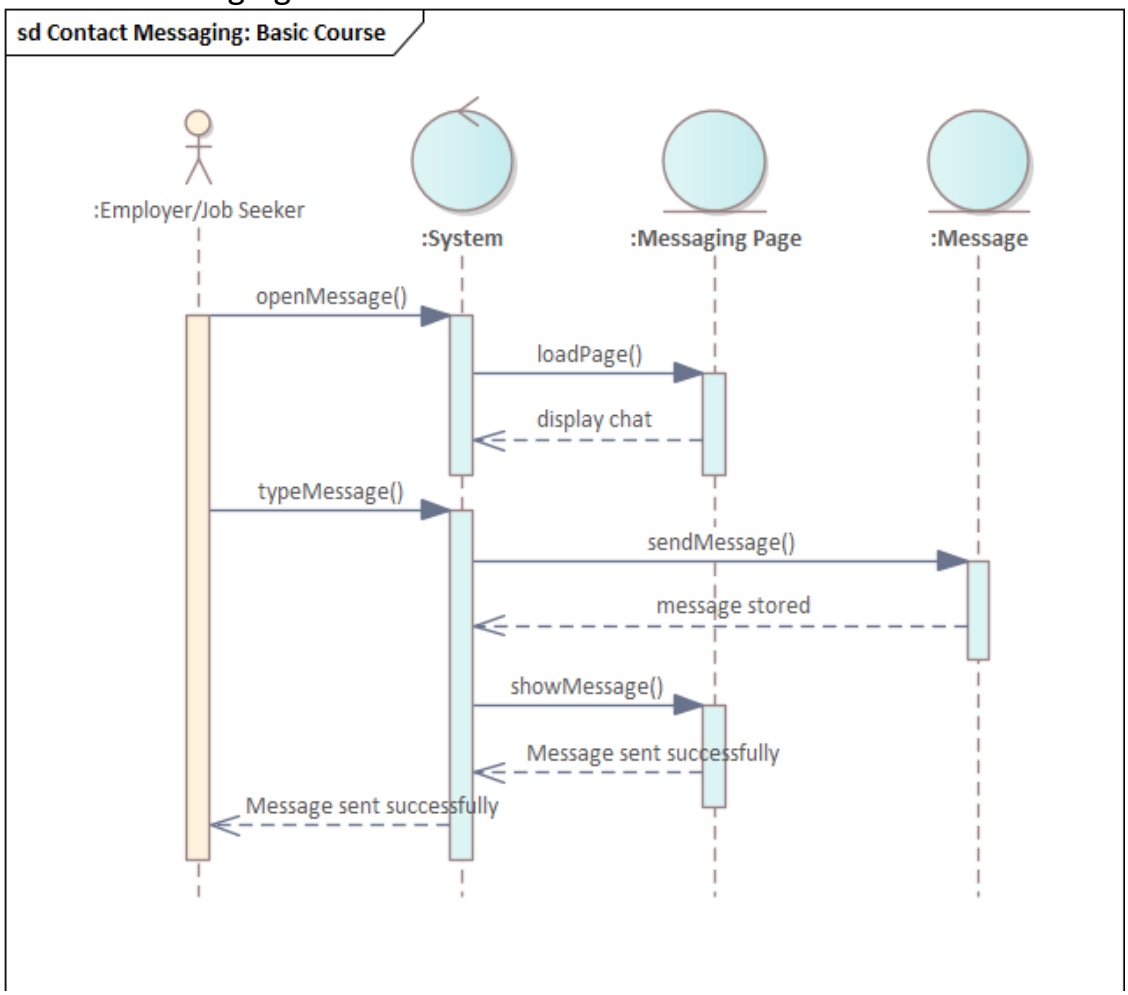
Create Job Posting: Alternative Course (Edit Before Submit)



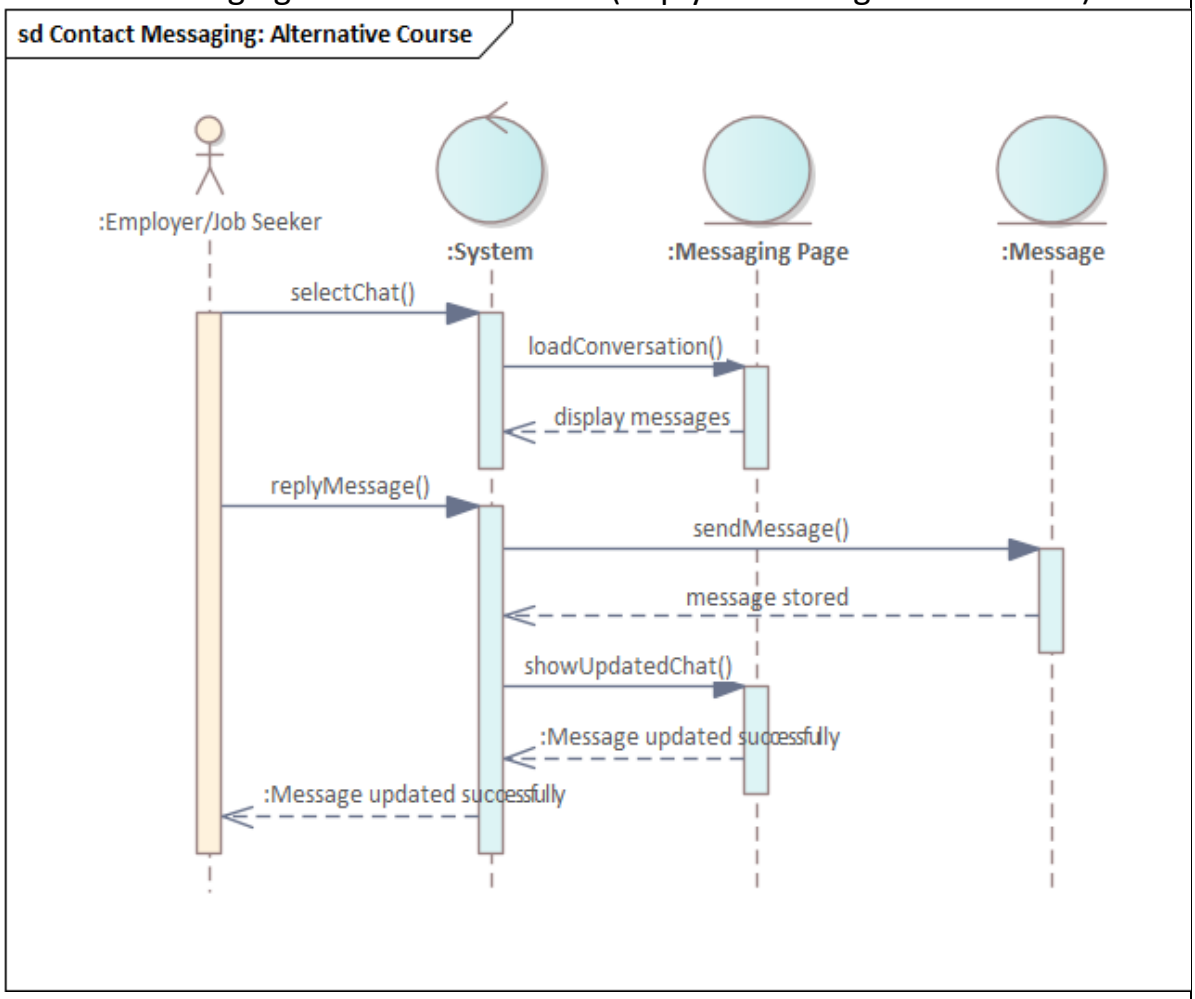
Create Job Posting: Exceptional Course (Invalid Input)



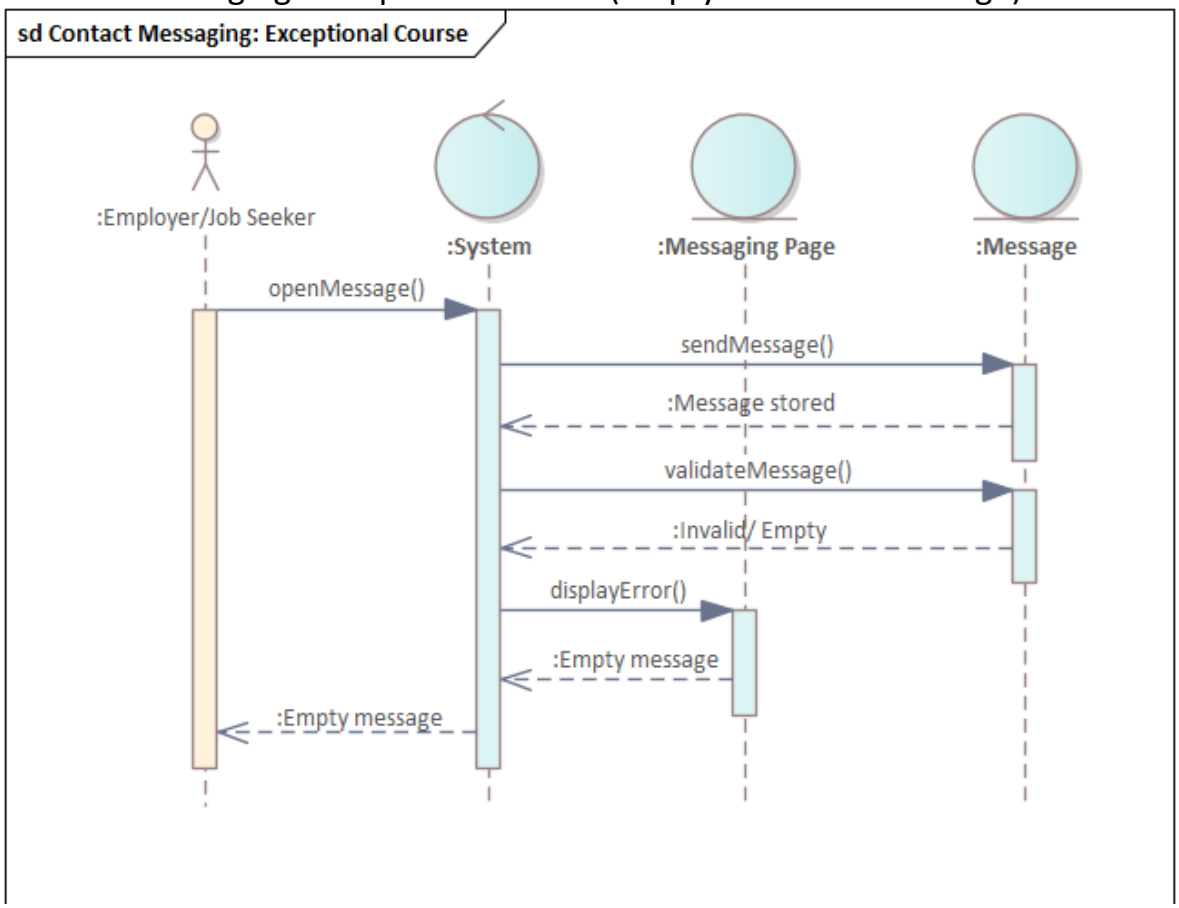
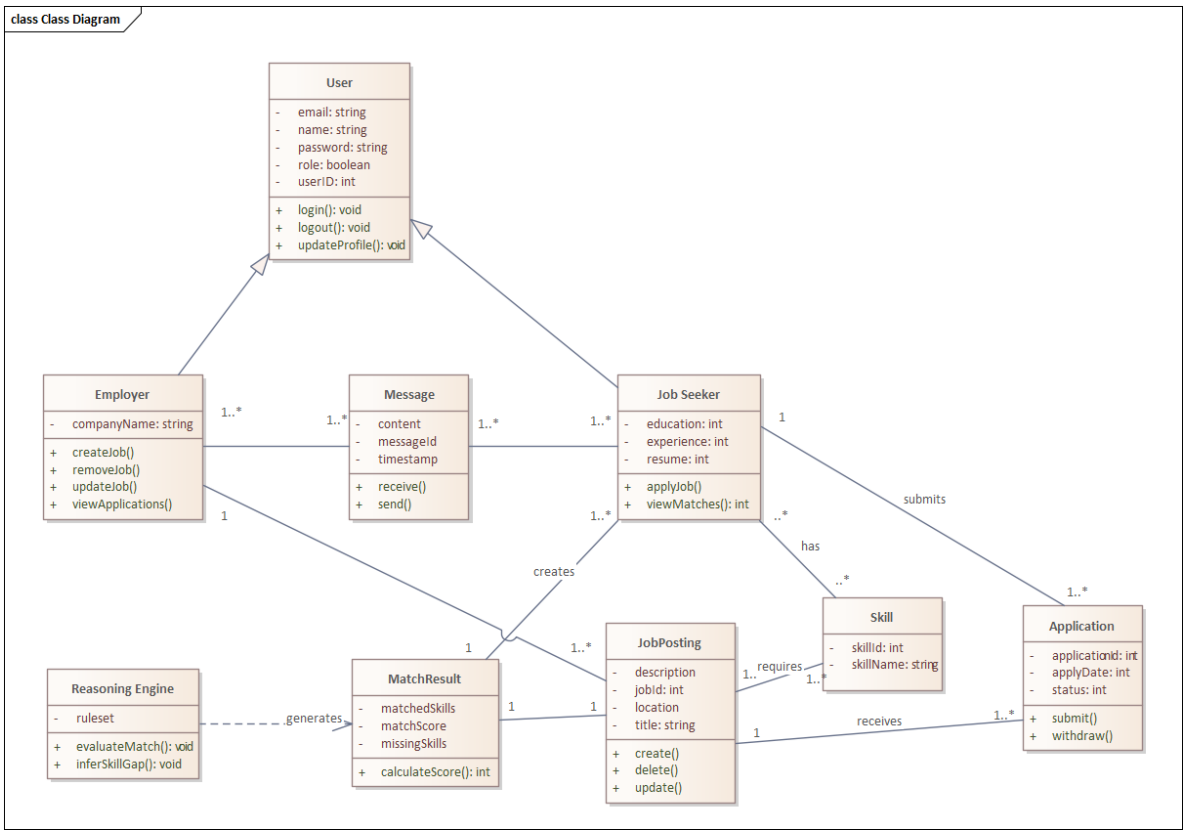
Contact Messaging: Basic Course



Contact Messaging: Alternative Course (Reply to Existing Conversation)



Contact Messaging: Exceptional Course (Empty or Invalid Message)



7. Class diagrams derived from sequence diagrams and your own additions